



Efi Arazi School
of Computer Science

Reichman University, Herzliya
Efi Arazi School of Computer Science
M.Sc. in Computer Science

Public Verification of Private Effort: Implementation and Analysis

by
Ariel Barak

Final project, submitted in partial fulfillment of the requirements for
the M.Sc. degree, School of Computer Science
Reichman University, Herzliya

May 2026

This work was carried out under the supervision of Prof. Tal Moran from the Efi Arazi School of Computer Science, Reichman University, Herzliya.

Abstract

This report presents the design and implementation of a full-stack web application realising the *Public Verification of Private Effort* (PPE) protocol, as formally introduced by Alberini, Moran, and Rosen [AMR14]. The protocol addresses a fundamental challenge in online polling: how can a pollster furnish any third-party verifier with a publicly auditable proof that every response was cast by a distinct participant who expended genuine effort, without relying on a trusted identity authority?

The application implements all six protocol phases—parameter announcement, registration, responder certification, poll response, results publication, and verification—as a client-server system backed by a FastAPI (Python) service and a React/TypeScript frontend. The certification graph is constructed deterministically from SHA-256 hashes over participant-pair identifiers [AMR14, Nat15], enabling any observer to reconstruct the ideal graph from publicly announced parameters alone and to audit for omitted, fabricated, or asymmetric edges. The Symmetric CAPTCHA scheme from Appendix B.3 of [AMR14] is adopted as the PPE mechanism, where each pair of peers exchanges arithmetic challenges and validates each other’s responses over a WebSocket signalling channel. Security parameters—edge probability p , effort threshold η_E , and validity threshold η_V —are derived from Theorem 4.4 of [AMR14], yielding a provable bound on an adversary’s ability to bias the poll outcome.

Contents

Abstract

1	Introduction	1
1.1	Research Objectives	1
2	Theoretical Background	3
2.1	The Public Verification of Private Effort Paradigm	3
2.2	Formal Definitions	3
2.3	The Responder Certification Graph	4
2.4	Security Properties and Parameter Selection	5
2.5	The Symmetric CAPTCHA Construction	5
3	System Architecture	7
3.1	Architectural Overview	7
3.2	Backend	7
3.3	Frontend	8
3.4	Communication Layer	9
3.5	Deployment	10
4	Protocol Implementation	11
4.1	Phase 1: Parameter Announcement	11
4.2	Phase 2: Registration	12
4.3	Phase 3: Responder Certification	12
4.4	Phase 4: Poll Response	13
4.5	Phase 5: Results Publication	14
4.6	Phase 6: Verification	14
5	Cryptographic Primitives and Graph Construction	16
5.1	Node Identity Derivation	16
5.2	The Deterministic Certification Graph	16

5.3	The η_E Exclusion Algorithm	17
5.4	Graph Reconciliation and Discrepancy Detection	17
5.5	The Math CAPTCHA PPE Mechanism	18
6	Security Analysis	19
6.1	Sybil Resistance	19
6.2	Pollster Integrity and Public Verifiability	20
6.3	Privacy	20
6.4	Implementation Limitations	21
7	Future Work	23

1 Introduction

Online polling has become a dominant channel for collective decision-making, yet it remains deeply susceptible to *Sybil attacks* [Dou02]—adversarial strategies in which a single actor creates many pseudonymous identities to cast additional votes and artificially distort the outcome. Standard countermeasures such as CAPTCHA verification protect against automated bots but offer no resistance to a determined human adversary registering dozens of browser sessions. Centralised identity schemes (e-mail confirmation, SMS codes, or OAuth integration) shift the trust problem to a third party and introduce privacy concerns for voters who wish to remain anonymous.

The *Public Verification of Private Effort* paradigm, introduced in [AMR14], offers a decentralised alternative grounded in the economics of human effort. The core intuition is simple: if registering a single vote requires a participant to engage in a non-trivial proof-of-effort with each of their assigned peers, then an adversary attempting to register k additional identities must invest k times the effort of an honest participant. Crucially, the protocol makes this effort *publicly verifiable*: the pollster publishes a certification graph and a transcript of all peer-to-peer interactions, and any third party can independently audit the record to detect manipulation—without any private keys or server-side state.

1.1 Research Objectives

This project aims to produce a faithful, working implementation of the PPE protocol that demonstrates all six protocol phases in a real, deployable web application. The implementation is guided by four concrete objectives drawn directly from the formal guarantees of [AMR14].

1. **Deterministic graph construction.** The certification graph must be constructed from publicly announced parameters using the SHA-

256 hash rule, so that no party needs to trust the pollster’s published edge list.

2. **Symmetric CAPTCHA instantiation.** The Symmetric CAPTCHA of [AMR14] must be realised as the bilateral PPE mechanism, with challenges and solutions exchanged over a real-time Web-Socket channel.
3. **Independent verification.** Any third party must be able to verify the poll outcome from the published results alone, without access to any private keys or server-side state.
4. **Realistic-scale demonstration.** The system must support a demonstration with at least fifty simulated participants, validating the protocol’s behaviour under realistic load.

2 Theoretical Background

2.1 The Public Verification of Private Effort Paradigm

Alberini, Moran, and Rosen [AMR14] formalise the problem of constructing polls that are simultaneously anonymous, publicly verifiable, and Sybil-resistant without a trusted identity provider. Their central contribution is the notion of a *Privately Verifiable Proof of Effort* (PPE): a two-party interactive protocol between a prover and a verifier in which successful completion certifies that the prover invested a non-trivial but bounded amount of effort. The key property distinguishing PPE from a simple puzzle is that the proof is *private*—the verifier is convinced, but no external party can verify the outcome from the transcript alone.

The public verification layer is achieved by having participants certify one another, forming a *certification graph* whose edges represent successfully completed PPE sessions. The pollster publishes this graph together with a record of all responses. Because the graph structure is determined by a publicly known function of the participant identifiers and the edge probability parameter p , any third party can reconstruct the expected graph independently and compare it against the published record, detecting omissions or fabrications.

2.2 Formal Definitions

Definition 2.2.1 (Privately Verifiable Proof of Effort [AMR14]). A pair of randomised interactive algorithms (P, V) is a (σ, ε) -PPE scheme if it satisfies the following two properties.

1. **σ -Completeness.** When both parties are honest, $\Pr[\langle P, V \rangle = 1] \geq \sigma$.

2. **ε -Soundness.** For any computationally unbounded cheating prover P^* , $\Pr[\langle P^*, V \rangle = 1] \leq \varepsilon$.

The parameters satisfy $0 < \varepsilon < \sigma \leq 1$.

Intuitively, completeness guarantees that an honest participant succeeds with high probability, while soundness guarantees that a lazy participant—one who has not genuinely engaged with the challenge—cannot fake success with probability more than ε . The gap $\sigma - \varepsilon$ determines how effectively the protocol distinguishes genuine effort from random guessing.

2.3 The Responder Certification Graph

The responder certification graph $G_c = (V, E)$ is built over the set V of registered responders. Unlike traditional random graphs, the edges are determined *deterministically* before the protocol begins, via a hash function applied to each pair of node identifiers.

Definition 2.3.1 (Deterministic edge rule [AMR14]). Given node identifiers i and j (with $i \neq j$) and an edge probability $p \in (0, 1)$, the edge (i, j) exists in G_c if and only if:

$$\text{SHA-256}(\min(i, j) \parallel \max(i, j)) \leq p \cdot 2^{256},$$

where $\parallel$ denotes string concatenation and $\min/\max$ are taken lexicographically. The symmetric ordering ensures that both endpoints independently compute the same hash value and reach the same conclusion about edge existence without any coordination.

The expected degree of each node under this rule is $(m - 1)p$, where m is the total number of registered participants—the same binomial degree distribution as in the classical Erdős–Rényi random graph $G(m, p)$ [ER59]. A key structural property required by the protocol is that the graph is *Large-Set Expanding* (LSE).

Definition 2.3.2 (Large-Set Expanding graph [AMR14]). A graph G is (α, β) -LSE if for every set $S \subseteq V$ with $|S| \geq \alpha|V|$, the neighbourhood $N(S)$ satisfies $|N(S) \setminus S| \geq \beta|S|$.

Lemma 4.2 of [AMR14] establishes that for sufficiently large m and appropriate choice of p , the random graph $G(m, p)$ is LSE with high probability.

2.4 Security Properties and Parameter Selection

The central security theorem bounds the adversary’s ability to influence the poll outcome. The adversary controls a set of corrupted nodes and may cause them to fail PPE challenges strategically, seeking to have honest nodes excluded alongside its own.

Theorem 2.4.1 (Soundness bound, Theorem 4.4 of [AMR14]). *Let G_c be an (α, β) -LSE graph on m nodes with expected degree d . Fix an effort threshold η_E and a validity threshold η_V . Then the adversary’s advantage—the ratio C^* of its effective vote weight to that of an honest participant—satisfies:*

$$C^* \leq 1 + \mathcal{O}\left(\frac{1}{\sqrt{m \cdot d \cdot (\sigma - \varepsilon)^2}}\right).$$

The probability that the adversary exceeds this bound is at most $2^{-\kappa}$ for security parameter κ .

This theorem provides the basis for the parameter recommendation endpoint implemented in the system, which computes p , η_E , η_V , and C^* for a given (m, κ) pair. Completeness is addressed in Theorem 5.2 of [AMR14]: under the Symmetric CAPTCHA instantiation, an honest coalition completes the full certification graph with probability approaching 1 as m grows, provided that the honest failure rate on individual PPE executions is below $1 - \sigma$.

2.5 The Symmetric CAPTCHA Construction

Appendix B.3 of [AMR14] proposes the *Symmetric CAPTCHA* as a concrete instantiation of the PPE scheme. In this construction, each peer plays both the role of prover and verifier simultaneously: peer A generates a CAPTCHA challenge c_A and sends it to peer B , while B simultaneously generates c_B and sends it to A . Both peers must solve the other’s challenge and return the solution; the PPE is considered successful if and only if both peers answer correctly.

To prevent each peer from adaptively choosing its answer after observing the other’s, the construction incorporates a hash-based commit-reveal step. After the two peers exchange challenges and each computes its solution, peer A commits to its solution by computing $H_A = \text{SHA-256}(s_A \| r_A)$ for a freshly sampled random nonce r_A , and sends H_A to peer B (and B does

likewise in the opposite direction). Only once both commitments have been exchanged do the peers reveal (s_A, r_A) and (s_B, r_B) respectively; each side then verifies that the revealed solution opens the committed hash before validating it against its own challenge. The protocol's soundness under this construction follows from the unpredictability of CAPTCHA solutions: a cheating peer who has not seen the challenge cannot solve it with probability better than ε . The concrete step-by-step realisation is given in Protocol 1 (Section 4.3).

3 System Architecture

3.1 Architectural Overview

The application follows a classic client-server architecture. The pollster—the party administering the poll—operates through the same web interface as the respondents, but with elevated permissions to advance the protocol phases. Both roles interact with the server via the same REST API; there is no network-level distinction between them. The server is intentionally designed to be a *passive* data store and phase controller. Responder identity keys are generated entirely in the browser and never transmitted to the server; the peer-to-peer challenge exchange during certification (Phase 3) likewise takes place client-side over the WebSocket relay. The one exception is the registration CAPTCHA (Phase 2): the server generates the arithmetic challenge and retains its expected solution for validation, discarding it once the client’s answer is checked.

The application is packaged as two Docker [Doc24] containers—one for the backend and one for the frontend—orchestrated via Docker Compose, requiring no host-level dependencies beyond a working Docker installation. The frontend is served as a static single-page application built with Vite [YV24], while the backend exposes an HTTP/WebSocket API at port 8000.

3.2 Backend

The backend is implemented in Python using the FastAPI [Ram24] framework, chosen for its native support for asynchronous request handling and WebSocket connections—both necessary for real-time peer signalling during the certification phase. The application is organised into four primary layers.

API layer. REST endpoints are grouped by protocol phase under the `/api/poll` route prefix, with sub-routes for registration (POST `/register`), certification (GET `/neighbors`, GET `/certification/graph`), response (POST `/vote`), and verification (POST `/verify`, which accepts a JSON body selecting `global` or `local` mode and an optional `node_id`). A separate WebSocket endpoint at `/ws/{session_id}` handles peer-to-peer signalling.

Crypto layer. The `app/crypto/` module provides the two core algorithmic components: `graph.py`, which implements the deterministic graph construction and η_E exclusion logic; and `verification.py`, which implements Protocol 6—the independent graph reconstruction and reconciliation algorithm.

PPE layer. The `app/ppe/` module defines an abstract `PPEBase` interface with `generate()` and `validate()` methods, and provides the concrete `MathCaptchaPPE` implementation. The abstraction makes it straightforward to substitute alternative PPE mechanisms—image-based CAPTCHAs, proof-of-work puzzles—without modifying the protocol logic.

Storage layer. The `app/storage/memory.py` module is a thread-safe in-memory store keyed on session identifiers. All session state—registered nodes, CAPTCHA challenges, certification edges, vote records, and published results—is maintained here under a reentrant lock. The use of an in-memory store is an explicit design choice for the prototype; migration to a persistent backend (e.g. PostgreSQL or Redis) is identified as future work.

3.3 Frontend

The frontend is a React [Met24] single-page application written in TypeScript, built with Vite and styled with Tailwind CSS. It operates in one of three modes, selectable at launch: *Pollster* (creates and manages the poll session), *Responder* (registers and participates in certification and voting), and *Verifier* (retrieves published results and runs the independent verification algorithm).

A key architectural decision is that all private-key material is generated and retained exclusively on the client. The browser’s Web Crypto API [Wat17] (SubtleCrypto) generates an ECDSA P-256 key pair marked as *extractable*, so that it can be structured-cloned into the per-origin

`IndexedDB` store `ppe-keystore` and survive a page refresh—without this, a responder who reloaded the browser during certification would lose their identity and be forced to re-register. The private key never leaves the browser; it is held both in memory and, across page reloads, in `IndexedDB`. Only the `SubjectPublicKeyInfo`-encoded public key is transmitted to the server during registration, where it serves as the node’s pseudonym; the node identifier is derived from it via SHA-256 [Nat15].

A dedicated `GraphVisualization` component renders the live state of the certification graph during the certification phase, showing each node and the verified/failed/pending status of its edges in real time.

3.4 Communication Layer

Protocol phases 1–2 and 4–6 are implemented as standard HTTP request-response interactions over the REST API. Phase 3 (responder certification) is the exception: because certification requires two peers to exchange challenges and solutions in real time, a WebSocket connection is established per participant at the start of the certification phase.

The WebSocket endpoint acts as a signalling relay; it does not interpret PPE message content but routes messages between the relevant peer pairs. Messages are structured as JSON objects with a `type` field and a payload. Client-to-server types drive peer coordination: `ppe.relay` asks the server to forward an opaque payload to a named peer, and `ppe.complete` reports the outcome of a bilateral PPE exchange. Server-to-client broadcast types announce phase transitions: `certification.edge_added` notifies all connected clients that a new certified edge has been recorded, and `results.published` signals that the poll results are available for retrieval. Within a `ppe.relay/ppe.message` envelope the inner payload carries its own `type`—`challenge`, `commitment`, `solution`, `signature`, or `error`—but the server does not parse these inner types; they are passed through unchanged. This blind-signalling design means a misbehaving server can only drop, duplicate, or reorder messages—it cannot fabricate inner payloads, because those are cryptographically bound to their originating peer. The client-side `useWebSocket` hook manages the connection lifecycle, including automatic reconnection with a two-second back-off and a heartbeat ping every thirty seconds to prevent idle disconnection.

3.5 Deployment

Both components are containerised with Docker. The backend container runs Uvicorn as the ASGI server; the frontend container uses Nginx to serve the pre-built static assets. A `docker-compose.yml` at the repository root coordinates the two containers and exposes them on host ports 3000 (frontend) and 8000 (API). Inside the frontend container Nginx serves the static SPA on port 80 (mapped to host port 3000) and reverse-proxies `/api/` and `/ws/` paths to the backend container, so in a Docker Compose deployment a client only nee

4 Protocol Implementation

The PPE protocol proceeds in six sequential phases, each mapped to a distinct set of API endpoints and UI views. The server enforces phase ordering through a session status field that follows the finite state machine:

`registration` $\rightarrow$ `certification` $\rightarrow$ `voting` $\rightarrow$ `results` $\rightarrow$ `closed`.

Requests presupposing a phase not yet reached are rejected with HTTP 400.

4.1 Phase 1: Parameter Announcement

The pollster initiates a session by submitting a `POST /api/poll/create` request carrying the poll questions, the edge probability p , the effort threshold η_E , the validity threshold η_V , and the PPE type. The server generates a unique session identifier (UUID4), creates an ephemeral ECDSA P-256 key pair, retains only the public verification key (VK) in the session record (the private key is dropped immediately after the request completes), and stores the session in the in-memory store. The response returns the session identifier and the VK, which is published as part of the session manifest; the VK is currently unused after publication, and signing the published results with the corresponding private key is identified as future work in Section 6.4.

To aid parameter selection, the system exposes a `GET /api/poll/params/recommend` endpoint that accepts the expected number of respondents and a security level (`low/medium/high`, corresponding to $\kappa = 40, 80, 128$ respectively) and returns the recommended values of p , η_E , η_V , and C^* , computed from Theorem 2.4.1. A `/params/validate` endpoint allows the pollster to check custom parameters against the theorem's requirements and receive a list of any detected violations.

4.2 Phase 2: Registration

Phase 2 implements a one-sided PPE: each prospective responder must solve a CAPTCHA challenge to register. The registration flow proceeds in two steps. First, the client fetches a CAPTCHA challenge from `GET /api/poll/{session_id}/captcha`. The server generates an arithmetic problem whose difficulty is derived from η_E via $difficulty = 1 - \eta_E$, and stores the challenge in the session store with a five-minute expiry token.

Second, the client solves the challenge and submits the solution together with its public key (pseudonym) via `POST /api/poll/{session_id}/register`. The server validates the solution, marks the token as consumed (preventing replay), and derives the node identifier as the first 16 hexadecimal characters of `SHA-256(public_key)`. A successful response includes the `node_id` and the registration position. A WebSocket notification is broadcast to all connected clients to update the live participant count.

4.3 Phase 3: Responder Certification

Certification is the most complex phase. Each registered node must execute a bilateral PPE with every neighbour assigned to it by the deterministic graph rule. The phase begins when the pollster advances the session status to `certification`.

A node first retrieves its neighbour list from `GET /api/poll/{session_id}/neighbors` (with its node id sent in the `X-Node-ID` header). The server computes the neighbours by iterating over all registered node IDs and applying the SHA-256 threshold test for each pair. The `detailed` variant additionally returns each neighbour's public key, enabling direct challenge construction.

For each neighbour, the two peers perform the Symmetric CAPTCHA protocol over the WebSocket channel. Protocol 1 describes the steps of this bilateral exchange.

The outcome of each bilateral PPE is submitted to the server via the certification report endpoint, which stores a directed `CertificationEdge` record for each (from_node, to_node) pair. The pollster may monitor progress through the `/certification/threshold` endpoint; the session may only advance to voting once the required fraction of nodes have completed all their assigned PPEs.

**Protocol 1: Symmetric CAPTCHA PPE
(Phase 3)**

- Step 1: Both peers A and B each generate an arithmetic challenge (c_A and c_B respectively).
- Step 2: A sends c_A to B ; simultaneously, B sends c_B to A .
- Step 3: A solves c_B to obtain solution s_A ; B solves c_A to obtain s_B .
- Step 4: *Commit.* A samples a fresh nonce r_A and sends the commitment $H_A = \text{SHA-256}(s_A \| r_A)$ to B ; B does likewise, sending $H_B = \text{SHA-256}(s_B \| r_B)$ to A .
- Step 5: *Reveal.* Once both commitments are received, A sends (s_A, r_A) to B ; B sends (s_B, r_B) to A . Each peer verifies that the revealed pair opens the commitment it received, then validates the solution against its own challenge.
- Step 6: *Sign.* If both validations pass, each peer signs the canonicalised edge label $\text{PPE} : \min(id_A, id_B) - \max(id_A, id_B) : \text{peerPublicKey}$ using ECDSA P-256 and sends the signature to the other peer.
- Step 7: Upon receiving a valid peer signature, each peer reports **verified=true** to the server for this directed edge, attaching the peer's ECDSA signature to the certification edge record. A peer that fails commitment verification or solution validation reports **verified=false**.

4.4 Phase 4: Poll Response

Once certification is complete, the session enters the voting phase. Each certified node submits its vote via a POST request to the vote endpoint. The vote payload contains the node's answers to each question, together with a self-signature generated with the node's ECDSA private key (over the canonicalised vote payload) and a collection of edge signatures gathered from peers during the certification phase. The current implementation stores these signatures with each vote and includes them in the published results, but the verification algorithm (Section 4.6) does not yet validate

them cryptographically—verification rests on the determinism of the edge rule and the **verified/failed** flags reported by peers. Signature validation is identified as a limitation in Section 6.4. The server enforces vote uniqueness: a node that has already submitted a vote cannot submit a second one. Votes are stored as `VoteRecord` objects keyed on `node_id`.

4.5 Phase 5: Results Publication

When the pollster advances the session to the results phase, the server assembles the full session record into a single JSON document—the *published results*. This document contains: the session identifier, the pollster’s public verification key, the question manifest, the session parameters (p , η_E , η_V , `ppe.type`), the ordered list of registered node IDs, all certification edges with their verified/failed flags and optional signatures, all submitted votes with self-signatures and collected peer signatures, and the publication timestamp.

Crucially, the published results contain no private keys and no server-side secrets. Every piece of information required to independently verify the poll is present in this document. The document is frozen at publication: subsequent requests return the same snapshot.

4.6 Phase 6: Verification

Verification distinguishes the PPE protocol from a conventional poll. Any party—not only poll participants—can perform verification by fetching the published results and running the verification algorithm locally.

Local verification. Implemented in `verify_local()`, this allows an individual responder to confirm that their own vote appears in the published results and that all edges incident on their node in the ideal graph are accounted for in the published certification graph.

Global verification. Implemented in `verify_global()`, this is the full audit, proceeding in ten steps:

- Step 1: Extract node IDs and parameters from the published results.
- Step 2: Reconstruct the ideal graph G_c from scratch using the SHA-256 hash rule (Definition 2.3.1).
- Step 3: Index the published edges for $O(1)$ lookup.

- Step 4: Reconcile G_c against the published graph, classifying each ideal edge as published-verified, published-failed, or omitted.
- Step 5: Detect *fabricated edges* (in the published graph but absent from G_c)—these cause an immediate **REJECT**.
- Step 6: Detect *missing nodes* (participants who voted but are absent from the published node list)—also an immediate **REJECT**.
- Step 7: Check edge symmetry across the published graph.
- Step 8: Run the η_E exclusion pass on the reconciled graph.
- Step 9: Check the η_V validity threshold—if more than η_V fraction of respondents are excluded, the poll is declared **INVALID**.
- Step 10: Tally votes from non-excluded nodes only.

The final verdict is one of **ACCEPT**, **INVALID**, or **REJECT**, matching the literal return values of `verify_global()`.

5 Cryptographic Primitives and Graph Construction

5.1 Node Identity Derivation

Each participant’s node identifier is derived deterministically from their public key. The function `compute_node_id` computes SHA-256 [Nat15] of the UTF-8 encoding of the public key string and returns the first 16 hexadecimal characters. The resulting 64-bit identifier space makes collisions negligible even for tens of thousands of participants.

Public keys are generated client-side using the browser’s `SubtleCrypto.generateKey()` API [Wat17] with the ECDSA algorithm and the P-256 curve. The resulting key pair is used to sign the vote record and to authenticate WebSocket messages during certification. The private key never leaves the browser; it is held in memory and, across page reloads, in the IndexedDB store `ppe-keystore`. Only the exported public key (in `SubjectPublicKeyInfo` format, base64-encoded) is transmitted to the server as the node’s pseudonym.

5.2 The Deterministic Certification Graph

The `determine_neighbors` function implements the edge rule from Definition 2.3.1. For each candidate neighbour, it constructs the edge key as the colon-separated concatenation of the lexicographically smaller and larger of the two node IDs, computes its SHA-256 hash, converts the 256-bit result to a Python integer, and compares it to the threshold $\lfloor M \cdot p \rfloor$, where $M = 2^{256} - 1$ is the maximum 256-bit value. The edge exists if and only if the hash value does not exceed the threshold.

Both the server and the client-side verifier use this same function, and because SHA-256 is deterministic and the ordering is canonical, they always

produce identical neighbour lists. This design has an important security consequence: the pollster cannot manipulate the graph structure after the fact. If the pollster attempts to publish a certification graph containing edges inconsistent with the SHA-256 rule, any verifier will detect the discrepancy in Step 5 of the global verification algorithm (Section 4.6) and reject the results.

5.3 The η_E Exclusion Algorithm

The `compute_exclusions` function implements the η_E exclusion pass. For each node in the certification graph, it computes the ratio of failed edges to total edges. If this ratio exceeds η_E , the node is added to the excluded set and its vote is not counted in the final tally. The function handles both the runtime format (a list of `CertificationEdge` objects) and the serialised format used in the published results (a list of dicts), making it usable both during the poll and during independent verification.

An important subtlety in the verification implementation concerns the treatment of omitted edges. If the pollster fails to report a PPE result for an edge that the ideal graph indicates should exist, the verification algorithm does not silently ignore the omission. Instead, it injects a *synthetic failed edge* for that pair, treating the omission as a failed PPE. This means the pollster cannot selectively hide failed certifications by not reporting them: omitting an edge has the same effect as reporting it as failed.

5.4 Graph Reconciliation and Discrepancy Detection

The `reconcile_graph` function performs the core comparison between the independently reconstructed ideal graph and the published graph. It first builds a set of all ideal edges (directed pairs) and an index of all published edges. It then walks each ideal edge: if the edge is present in the published index, its reported verified/failed status is preserved; if it is absent, a synthetic failed edge is injected. After processing all ideal edges, it scans the published index for any edge not present in the ideal edge set—these are *fabricated edges*, which cause an immediate REJECT.

The `GraphDiscrepancies` dataclass aggregates the results of this reconciliation: omitted edges, fabricated edges, asymmetric edges, missing nodes, extra nodes, and unverified edges. Critical issues (fabricated edges and

missing nodes) are distinguished from non-critical anomalies and trigger an immediate REJECT rather than merely reducing the count of valid votes.

5.5 The Math CAPTCHA PPE Mechanism

The application instantiates the PPE scheme as an arithmetic CAPTCHA. The `MathCaptchaPPE` class extends the abstract `PPEBase` interface and generates random arithmetic problems of the form $a \oplus b = ?$, where $\oplus \in \{+, -, \times\}$ and the operand range is determined by the difficulty parameter. The difficulty is derived from η_E as $difficulty = 1 - \eta_E$: a strict threshold (small η_E) yields harder problems (larger operands and multiplication), while a lenient threshold yields easier ones.

The `MathCaptchaPPE.validate()` method re-parses the challenge string and recomputes the expected answer from it—supporting stateless validation in any context where the challenge text is available. During the *registration* flow, however, the server takes a simpler path: it stores the canonical solution alongside the challenge in a `CaptchaChallenge` record keyed by a one-time token that expires after five minutes, and compares the submitted answer directly against the stored value. The PPE certification flow (Phase 3) uses `validate()` to re-derive the expected answer locally for each peer’s challenge without consulting stored state. In both flows the token is marked consumed immediately upon successful validation, preventing replay attacks.

6 Security Analysis

6.1 Sybil Resistance

The protocol’s primary security goal is resistance to Sybil attacks [Dou02]. An adversary who wishes to register k fake identities in addition to a single honest identity must:

1. Solve k CAPTCHA challenges during registration (Phase 2); and
2. Conduct bilateral PPE sessions with all graph neighbours of each fake node during certification (Phase 3).

Because the neighbour assignments are determined by the SHA-256 hash rule—which the adversary cannot manipulate—each fake node is assigned an expected degree of $(m - 1)p$ neighbours. If a fake node fails too many of its PPE sessions (more than η_E), it is excluded and its vote is discarded. If the adversary passes enough sessions for its fake nodes to survive exclusion, it has invested proportional effort for each—precisely the intent of the PPE mechanism.

The residual advantage available to the adversary—the C^* bound from Theorem 2.4.1—quantifies how much extra weight the adversary’s votes carry relative to an honest participant’s, accounting for the fact that excluded honest nodes reduce the effective vote count. For the parameter set recommended at security level medium ($\kappa = 80$, $m = 1000$), the implementation computes $\eta_V \approx 0.015$ and $C^* \approx 1.03$, meaning the adversary can shift the effective vote count by at most approximately 3% above its proportional share. Tighter guarantees are obtained at higher security levels: at $\kappa = 128$, $m = 1000$ the system yields $C^* \approx 1.011$.

6.2 Pollster Integrity and Public Verifiability

A distinct threat model concerns not the adversarial participant but the adversarial pollster. A malicious pollster might attempt to manipulate the results by:

1. Selectively publishing only a subset of certification edges, causing more honest nodes to be excluded;
2. Adding fabricated edges to rescue excluded dishonest nodes; or
3. Omitting certain votes from the published results.

The verification algorithm’s reconciliation step directly addresses all three attacks. Omitted edges are treated as failures, so selective omission of failed edges provides no advantage to the pollster. Fabricated edges are detected immediately and cause a REJECT. Missing votes are also classified as critical issues triggering a REJECT. Any individual participant can run local verification to confirm that their own vote is present, and any third party can run global verification to audit the complete result set.

In addition to the pollster-manipulation attacks listed above, the `/publish` endpoint accepts an optional `X-Pollster-Key` header which, when supplied, is compared byte-for-byte against the session’s stored verification key; mismatches are rejected with HTTP 403. This mitigates a third party who has obtained a session identifier from attempting to publish a fabricated result set on the pollster’s behalf. The header is currently optional for backwards compatibility (omitting it triggers a server-side warning but does not block publication); making it mandatory and signing the published results with the corresponding private key (Section 6.4) would close the residual gap.

6.3 Privacy

Participant privacy is protected at two levels. At the identity level, participants are represented only by their pseudonymous `node_id` (derived from their public key), and no real-world identity information is collected or stored. At the vote level, votes are associated with node IDs in the published results, making the vote record pseudonymous but not fully anonymous. Full vote anonymity would require an additional cryptographic layer (e.g. a mix-net or threshold decryption scheme), which is beyond the scope of the current implementation but is noted as future work.

6.4 Implementation Limitations

The current implementation has five security gaps that a production deployment should address.

In-memory storage. The in-memory storage layer does not persist across server restarts, making the system unsuitable for long-running polls where server availability cannot be guaranteed.

Persistent private-key storage. Because key pairs are kept in `IndexedDB` so that responders survive a page refresh, the private key is exposed to any code with access to the browser’s `IndexedDB` API—for example, a compromised extension or a second user of the same browser profile. A higher-assurance deployment would generate keys as non-extractable and accept that a page refresh ends the session, relying on the opaque Web Crypto key handle for isolation.

Unsigned published results. The server-generated key pair is used only to export a public verification key in the session manifest; the private key is discarded, and the VK is not yet used to sign the published results. A man-in-the-middle who could replace the published results JSON would therefore not be detected at the cryptographic level. Implementing a server signature over the published results would close this gap.

Unchecked vote signatures. Each vote is published alongside a self-signature and a list of peer signatures gathered during certification, but neither `verify_global()` nor `verify_local()` calls `verify_signature()` against any of them. Verification therefore rests entirely on the deterministic SHA-256 edge rule and the `verified/failed` flags reported by peers. A pollster willing to fabricate signature strings cannot evade the edge-rule check, but can publish technically inconsistent signatures without consequence. Wiring `app/crypto/signatures.py::verify_signature` into the reconciliation loop in `verify_global()` would close this gap.

Absent challenge-binding signature. An earlier version of the Symmetric CAPTCHA implementation attached an ECDSA binding signature over the sorted concatenation of the two peers’ public keys to each challenge payload, which would prevent a man-in-the-middle from re-using a captured challenge between two other peers. This binding was removed during

development; as a result, peer-to-peer challenge proxying is not currently prevented. Reintroducing the binding signature is identified as a direction for future work.

Weak CAPTCHA instantiation. The arithmetic CAPTCHA is a weak PPE instantiation for a determined adversary with computational resources. Stronger mechanisms—image recognition, memory-hard puzzles, or interactive knowledge-verification games—are preferable for high-stakes deployments.

7 Future Work

The current implementation constitutes a complete, working prototype of the PPE protocol. Several extensions are planned or identified as priorities for the next development phase.

Persistent storage. The most immediately impactful improvement is replacing the in-memory storage layer with a persistent database backend (e.g. PostgreSQL), enabling the system to support long-duration polls and to recover gracefully from server restarts without losing session state. A persistent store would also open the door to richer audit logging and to post-hoc forensic analysis of certification graphs.

Stronger PPE mechanisms. The abstract PPEBase interface was designed precisely to make PPE substitution straightforward. Image-based CAPTCHA challenges, memory-hard proof-of-work puzzles, or interactive knowledge-verification games could be plugged in without modifying any protocol logic. A systematic comparison of different PPE instantiations with respect to the completeness and soundness parameters (σ, ε) and their downstream effect on C^* would be a valuable empirical contribution.

Pluggable proof-of-effort mechanisms. The abstract PPEBase interface makes it straightforward to extend the system with richer proof-of-effort mechanisms beyond the current arithmetic CAPTCHA. A plugin architecture that allows new PPE instantiations—image-based challenges, memory-hard puzzles, or domain-specific knowledge games—to be registered and selected at session-creation time would let the pollster choose the mechanism best suited to the expected participant population and threat model. A systematic comparison of different instantiations with respect to the (σ, ε) parameters and their downstream effect on C^* would be a valuable empirical contribution.

Enhanced dynamic interface. The current frontend provides a functional but minimal view of the certification phase. Richer real-time feedback—animated graph layout updates as edges are certified, live histograms of per-node success rates, and an at-a-glance exclusion counter—would make the protocol’s progress tangible for non-expert participants and would support interactive demonstrations with larger cohorts. Extending the verifier view to display a side-by-side diff of the expected and observed certification graphs would similarly lower the barrier to independent auditing.

Signed published results. Adding a cryptographic signature over the published results JSON—using the server’s ECDSA key pair generated at session creation—would allow any verifier to confirm that the results have not been tampered with in transit. This would close the man-in-the-middle gap identified in Section 6.4 and constitutes a straightforward extension requiring only a small addition to the results-publication endpoint and a corresponding verification helper on the client.

Large-scale simulation. A scripted simulation with

Bibliography

- [AMR14] Gilad Alberini, Tal Moran, and Alon Rosen. Public verification of private effort. In *Theory of Cryptography – 11th Theory of Cryptography Conference, TCC 2014*, volume 8349 of *Lecture Notes in Computer Science*, pages 432–453. Springer, 2014.
- [Doc24] Docker, Inc. Docker: Accelerated container application development. Online, 2024.
- [Dou02] John R. Douceur. The Sybil attack. In *Peer-to-Peer Systems, First International Workshop, IPTPS 2002*, volume 2429 of *Lecture Notes in Computer Science*, pages 251–260. Springer, 2002.
- [ER59] Paul Erdős and Alfréd Rényi. On random graphs I. *Publicationes Mathematicae*, 6:290–297, 1959.
- [Met24] Meta Platforms, Inc. React: The library for web and native user interfaces. Online, 2024.
- [Nat15] National Institute of Standards and Technology. FIPS PUB 180-4: Secure Hash Standard (SHS). Federal Information Processing Standards Publication, 2015.
- [Ram24] Sebastián Ramírez. FastAPI: Modern, fast web framework for building APIs with Python. Online, 2024.
- [Wat17] Mark Watson. Web Cryptography API. W3C Recommendation, 2017.
- [YV24] Evan You and Vite Contributors. Vite: Next generation frontend tooling. Online, 2024.